User-defined methods

CSC103 Programming Fundamentals / Lecture 15

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Define static double average(int a, int b). Call it with 60 and 81 and print the result.

Define a static method with two integer parameters and a double result.

Learning objectives

- Define and call reusable methods
- Choose parameters and return types
- Separate computation from console output

CLO-2
A method definition
Calls and returned values
Void methods
Method contracts

A method definition

- A method header names its return type and parameters.
- The body defines the computation.
- A non-void method must return a compatible value on every normal path.

Example

```
static int square(int value) {
  return value * value;
}
```

Calls and returned values

- The caller evaluates arguments and passes their values.
- return ends the current method invocation.
- The caller can store, print or combine the result.

Example

```
int area = square(5);
System.out.println(square(3) + 1);
```

Void methods

- A void method performs an action without returning a value.
- return; can end a void method early.
- A caller cannot assign a void result to a variable.

Example

```
static void greet(String name) {
  System.out.println("Hello " + name);
}
```

Method contracts

- Choose parameters that represent the required inputs.
- State preconditions and what the return value means.
- Keep one clear responsibility per method.

Example

```
Contract: rectangleArea(width,height)
Inputs: nonnegative side lengths
Result: product in square units
```

A result is reusable when calculation stays separate

- A calculation method returns a value instead of deciding how it will appear on screen.
- The caller can print, store or test that result.
- Keeping console input outside the method also permits deterministic tests.

Example

```
average(60,81) returns 70.5.
The caller decides whether to show one or two decimal places.
```

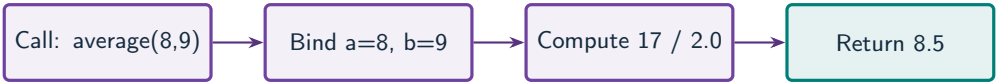
Correct types matter inside the method

- The declared return type does not change integer arithmetic inside an expression.
- If a and b are int, (a+b)/2 truncates before returning.
- Convert before arithmetic that must retain a fraction.

Example

```
return ((double) a + b) / 2;
The sum and division use double arithmetic.
```

A method is an input-output contract



What to notice

The caller supplies values; the method returns a result that the caller can reuse.

Key distinctions

Part	Example	Role
Return type	double	Type of result
Method name	average	Operation identifier
Parameters	int a, int b	Local inputs
Return expression	((double)a+b)/2	Computed result

These distinctions determine which operation or control structure fits the problem.

First example: methods

```
static int rectangleArea(int width, int height) {
  return width * height;
}
```

First example: execution

```
int result = rectangleArea(5, 3);
System.out.println(result);
```

First example: result

15
width receives 5 and height receives 3.
return sends their product to the caller.

Void methods
Method contracts

Guided practice

Define static double average(int a, int b). Call it with 60 and 81 and print the result.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Define a static method with two integer parameters and a double result.
- Convert a to double before addition and division.
- Call the method from main and print the returned result.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution15 {
  public static void main(String[] args) throws Exception {
    double result = average(60, 81);
    System.out.println(result);
  }
  static double average(int a, int b) {
    return ((double) a + b) / 2;
  }
}
```

Execution trace

Execution point	State	Result
Call in main	Arguments 60, 81	Invocation starts
Parameter binding	a=60, b=81	Copies of arguments
Return calculation	141.0 / 2	70.5
Caller assignment	result receives return	70.5

Follow the changing state in execution order.

Solution result

70.5

Why this result follows
Call the method from main and print the returned result.

A common incorrect approach

```
static int larger(int a, int b) {
if (a > b) return a;
}
```

Example

The path `a <= b` lacks a `return`. Add `return b` after the `if`.

Boundary and failure checks

Check the stated input assumptions.
Write isEven(int n) returning boolean. Demonstrate calls for -2, 0, 3 and 8.

return ((double) a + b) / 2;
Expected: 70.5. Casting before addition also avoids int overflow during a+b for extreme integer inputs.

Check your understanding

How do a parameter and an argument differ? Can a void call appear on the right of an assignment?

Write a prediction and explain the rule that supports it.

Answer and explanation

A parameter is declared in the method. An argument is supplied by the caller. A void call provides no value for assignment.

The path a <= b lacks a return. Add return b after the if.

Compare cases and predict outcomes

Arguments	Correct mean	Reason to test
8, 9	8.5	Fractional answer
0, 0	0.0	Zero case
-4, 4	0.0	Opposite signs

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Write a double-returning average method for two ints. Prevent integer addition overflow before dividing.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
static double average(int a, int b) {
  return ((double) a + b) / 2.0;
}
```

- Casting a before addition makes the addition use double arithmetic.
- Casting after a+b would occur too late if the int sum overflowed.
- The method returns a value and leaves output formatting to its caller.

Lecture summary

- and call reusable methods
- parameters and return types
- Separate computation from console output

- Define a static method with two integer parameters and a double result.
- Convert a to double before addition and division.
- Call the method from main and print the returned result.

After-class practice

Write isEven(int n) returning boolean. Demonstrate calls for -2, 0, 3 and 8.

Syllabus reading
Deitel Ch 6
Next lecture: Parameters, stack and scope